Data structures

LINEAR-TIME SORTING

This Week

- Linear-time sorting
  - Counting sort
  - Bucket sort
  - Radix sort

Reminder - Linear-Time Sorting

- Comparison-based sorting algorithms are most appropriate when we have **no prior** information about the array that requires sorting.
- If there is no prior knowledge about the keys, then the average and worst-case running times are $\Omega(n \cdot \log n)$.
- Knowing more about the elements can sometimes help us to sort faster.
- Remark: We saw in the last recitation that any sorting algorithm requires $\Omega(n)$ time.

Reminder - Counting Sort

Assumptions:

- The keys are Integers in $[1, k]$ for some parameter k.

Counting Sort

```
COUNTING-SORT(A, n, k)
 1  let B[1 : n] and C[0 : k] be new arrays
 2  for i = 0 to k
 3      C[i] = 0
 4  for j = 1 to n
 5      C[A[j]] = C[A[j]] + 1
 6  // C[i] now contains the number of elements equal to i.
 7  for i = 1 to k
 8      C[i] = C[i] + C[i − 1]
 9  // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```

Counting Sort – Time complexity

The for-loop of lines 2-3 takes $\Theta(k)$

The for-loop of lines 4-5 takes $\Theta(n)$

The for-loop of lines 7-8 takes $\Theta(k)$

The for-loop of lines 11-13 takes $\Theta(n)$

Overall: $\Theta(n+k)$

We use counting sort when $k = O(n)$

Therefore, overall running time: $\Theta(n)$

```
COUNTING-SORT(A, n, k)
 1  let B[1 : n] and C[0 : k] be new arrays
 2  for i = 0 to k
 3      C[i] = 0
 4  for j = 1 to n
 5      C[A[j]] = C[A[j]] + 1
 6  // C[i] now contains the number of elements equal to i.
 7  for i = 1 to k
 8      C[i] = C[i] + C[i − 1]
 9  // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```

Radix Sort

Assumptions:

- Each key consists of a constant number of fields (e.g., integers with a constant number of digits).
- Each field can be stably sorted in linear time (e.g., satisfies the requirements of counting sort).

- Start by sorting the numbers according to the least significant digit, then proceed to sort according to the next digit up to the most significant digit.

Radix Sort

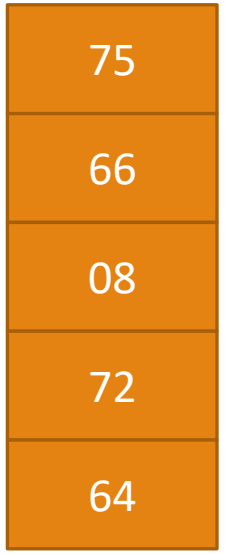
```
RADIX-SORT(A, n, d)
1  for i = 1 to d
2      use a stable sort to sort array A[1 : n] on digit i
```

Although the pseudocode for RADIX-SORT does not specify which stable sort to use,

COUNTING-SORT is commonly used.

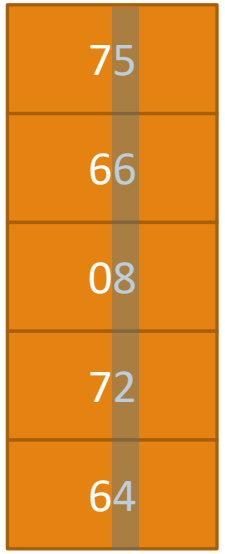
Radix Sort

```
RADIX-SORT(A, n, d)
1  for i = 1 to d
2      use a stable sort to sort array A[1 : n] on digit i
```



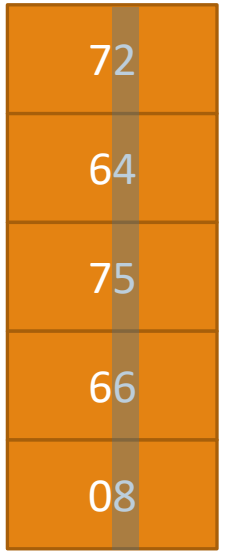
Radix Sort

```
RADIX-SORT(A, n, d)
1  for i = 1 to d
2      use a stable sort to sort array A[1 : n] on digit i
```



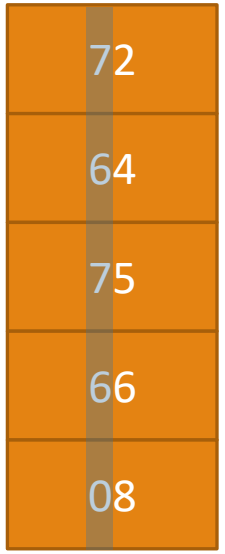
Radix Sort

```
RADIX-SORT(A, n, d)
1  for i = 1 to d
2      use a stable sort to sort array A[1 : n] on digit i
```



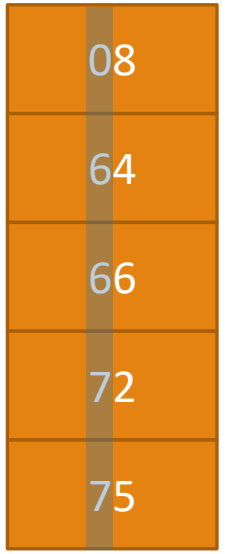
Radix Sort

```
RADIX-SORT(A, n, d)
1  for i = 1 to d
2      use a stable sort to sort array A[1 : n] on digit i
```



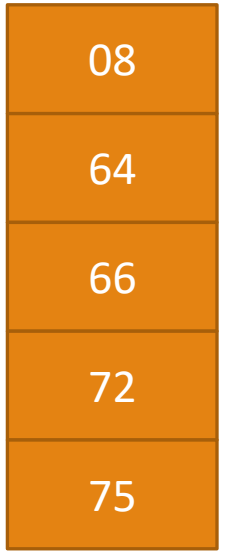
Radix Sort

```
RADIX-SORT(A, n, d)
1  for i = 1 to d
2      use a stable sort to sort array A[1 : n] on digit i
```



Radix Sort

```
RADIX-SORT(A, n, d)
1  for i = 1 to d
2      use a stable sort to sort array A[1 : n] on digit i
```



Digits

Digits help represent a quantity as a sum of smaller quantities.

In decimal:

1234 =

Digits

Digits help represent a quantity as a sum of smaller quantities.

In decimal:

1234 = 4*1 + 3*10 + 2*100 + 1*1000

Representing numbers in different bases

The meaning of the number 1234 (in decimal) means:

```
1234 =
1000 +
 200 +
  30 +
   4 =
1*1000 + 2*100 + 3*10 + 4*1 =
```

$$= 1 * 10^3 + 2 * 10^2 + 3 * 10^1 + 4 * 10^0$$

Representing numbers in different bases

How do we express the number 1234 (in decimal) in base 2?

Representing numbers in different bases

Modulo 2 and divide by 2:

1234 → 0

617 → 1

308 → 0

154 → 0

77 → 1

38 → 0

19 → 1

9 → 1

Representing numbers in different bases

Modulo 2 and divide by 2:

1234 → 0

617 → 1

308 → 0

154 → 0

77 → 1

38 → 0

19 → 1

9 → 1

4 → 0

2 → 0

1 → 1

0

Representing numbers in different bases

Modulo 2 and divide by 2:

1234 → 0

617 → 1

308 → 0

154 → 0

77 → 1

38 → 0

19 → 1

9 → 1

4 → 0

2 → 0

1 → 1

0

Collect the remainders:

10011010010

Representing numbers in different bases

Modulo 2 and divide by 2:

1234 → 0

617 → 1

308 → 0

154 → 0

77 → 1

38 → 0

19 → 1

9 → 1

4 → 0

2 → 0

1 → 1

0

Collect the remainders:

10011010010

$= 0*2^0 + 1*2^1 + 0*2^2 + 0*2^3 \ldots + 1*2^{10}$

Representing numbers in different bases

Modulo 2 and divide by 2:

1234 → 0

617 → 1

308 → 0

154 → 0

77 → 1

38 → 0

19 → 1

9 → 1

4 → 0

2 → 0

1 → 1

0

Collect the remainders:

10011010010

$= 0*2^0 + 1*2^1 + 0*2^2 + 0*2^3 \ldots + 1*2^{10}$

$= 0*1 + 1*2 + 0*4 + 0*8 \ldots + 1*1024$

Computing the digits of integer $x > 0$ in base b

$digits$ = array of size $\lceil \log_b x \rceil$, initialized to zeros.

$i = 0$

$while\ x\ >\ 0\ \{$

$\quad digits[i]\ = x \bmod b$

$\quad x = \lfloor x/b \rfloor$

$\quad i\ =\ i + 1$

$\}$

Time complexity: $O(\log_b x)$

Representing numbers in different bases

Decimal = base 10

Binary = base 2

Octal = base 8

Hexadecimal = base 16

Any natural number can be used as a base.

Radix Sort – Time complexity

- Suppose the numbers we sort are
  - Represented in base k and
  - Each number consists of at most d digits.
- Complexity:
  - Counting sort according to each digit runs in $O(n + k)$.
  - The total runtime is $O(d \cdot (n + k))$.

- If $k = O(n)$ and $d = O(1)$ then radix sort takes $O(n)$ time.

Question 1

- You are given an array with n integers in the range $[0, n^3 - 1]$.
  Each integer is stored in a single word (remember the Word-RAM model!).

- Suggest an efficient way to sort the array.

- Analyze the worst-case runtime of the following options:
  - Comparison-based sorting
  - Counting sort
  - Radix sort

Question 1 – Solution

- Comparison based sorting algorithms take $\Omega(n \log n)$.
- Counting sort would take $O(n + n^3) = \mathrm{O}(\mathrm{n}^3)$.
- Radix sort with base 10 would run in $O\left(\log_{10}(n^3)\,(n + 10)\right) = O(n \log n)$ time.
- How about Radix sort with base n?

Question 1 – radix sort base n

- Each number has $d = \log_n(n^3) = 3$ "digits" when represented base $\mathrm{k} = n$
- Compute digits of each number in base $\mathrm{k} = n$ in $O(3n) = O(n)$ time
- Use radix sort in $O(d(n+k) = O(\log_n(n^3)(n+n)) = O(n)$
- So, total time is $O(n)$

Question 3 – radix sort base n

- Assume n is a power of 2 (if not work with the closest power of 2)
- Convert the input from base 2 to base n by treating each $\log_2 n$ consecutive bits as a single “digit”
- So, we have $\log_n n^3 = 3$ “digits”

$$\underbrace{1101110010}_{\log n}\ \underbrace{1000111110}_{\log n}\ \underbrace{0101010111}_{\log n}$$

- So, the running time is $O(\log_n(n^3)(n+n)) = O(n)$

Reminder - Bucket Sort

- Assumption:
  - Elements are distributed uniformly in $[0,1)$.

- Bucket Sort:
  - Partition the interval $[0,1)$ into n 'buckets' of size $\frac{1}{n}$.
  - Insert each element from the array into its bucket (can use linked lists to represent each bucket).
  - Sort each bucket separately.
  - Concatenate the buckets back according to their order into a sorted array.

Bucket Sort

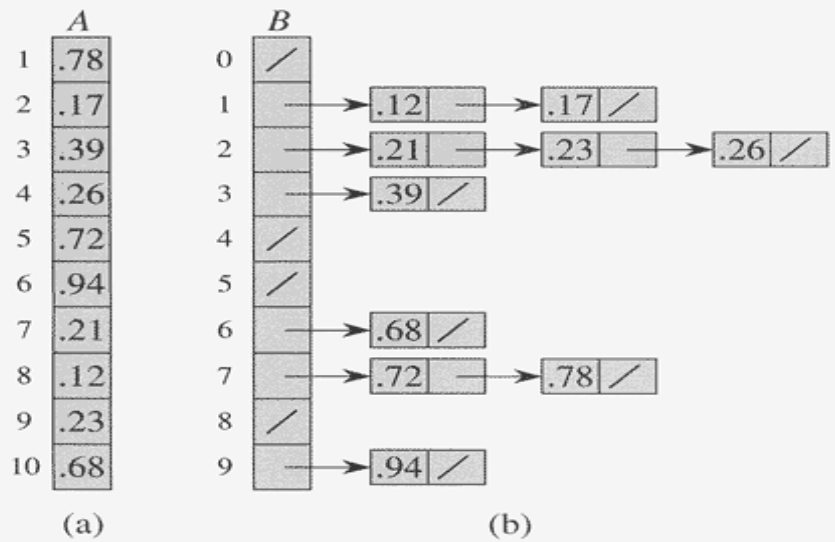


Figure 8.4 The operation of BUCKET-SORT. **(a)** The input array $A[1 \ldots 10]$. **(b)** The array $B[0 \ldots 9]$ of sorted lists (buckets) after line 5 of the algorithm. Bucket i holds values in the half-open interval $[i/10, (i + 1)/10)$. The sorted output consists of a concatenation in order of the lists $B[0], B[1], \ldots, B[9]$.

Bucket Sort

```
BUCKET-SORT(A)
1  n ← length[A]
2  for i ← 1 to n
3        do insert A[i] into list B[⌊nA[i]⌋]
4  for i ← 0 to n − 1
5        do sort list B[i] with insertion sort
6  concatenate the lists B[0], B[1], . . . , B[n − 1] together in order
```

Bucket Sort - complexity

- Inserting all the elements into the buckets will take $O(n)$.
- Concatenating the buckets can be done in $O(n)$.
- The running time is dominated by the time to sort the elements in every bucket.
- **Best-case** $O(n)$
- **Worst-case** $O(n^2)$ ($O(n \log n)$ if using merge-sort instead of insertion sort)
- **Average-case** $O(n)$

Question 2

- Suppose you are given n real numbers with the following assumption:
  - Half of the numbers are uniformly distributed in $[0,1)$.
  - The other half is uniformly distributed in $[1,5)$.
- How can we use bucket sort to sort the numbers in the most efficient way?

Question 2 - Solution

- We partition the interval $[0,5)$ into n unequal buckets.
- For $0 \leq i < \frac{n}{2}$, bucket i is for the range $\left[\frac{2i}{n}, \frac{2(i+1)}{n}\right)$.
- These first $\frac{n}{2}$ buckets partition the interval $[0,1)$ into $\frac{n}{2}$ equal parts.
- For $0 \leq i < \frac{n}{2}$, bucket i is for the range $\left[1 + 4\frac{2i}{n}, 1 + 4\frac{2(i+1)}{n}\right)$.
- These last $\frac{n}{2}$ buckets partition the interval $[1,5)$ into $\frac{n}{2}$ equal parts.
- Now the elements are distributed uniformly across the buckets, so the running time of bucket sort holds.

Sorting

Algorithm	Worst-case running time	Average-case running time
Insertion sort	$\Theta(n^2)$	$\Theta(n^2)$
Merge sort	$\Theta(n \lg n)$	$\Theta(n \lg n)$
Heapsort	$\Theta(n \lg n)$	$\Theta(n \lg n)$
Quicksort	$\Theta(n^2)$	$\Theta(n \lg n)$
Counting sort	$\Theta(k + n)$	$\Theta(k + n)$
Radix sort	$\Theta(d(n + k))$	$\Theta(d(n + k))$
Bucket sort	$\Theta(n^2)$	$\Theta(n)$

Question 3

Describe an $O(n)$-time algorithm that takes as input an array A of n distinct numbers and a positive integer $k \leq n$. Assume n is odd.

The algorithm finds the k numbers in A that are closest to the median of A.

The distance between two numbers x, y is defined to be $|x - y|$.

Question 3 – Solution

1. Find the median of the array in $O(n)$ using the median of medians algorithm.
2. Go over the input array, and for each element, calculate the distance to the median.
3. Store the distances in a new array.
4. Find the $k'th$ order statistic in the new array, again in $O(n)$ using the median of medians algorithm.
5. Output all numbers in the original array whose distance to the median is smaller than the $k'th$ order statistic.

Question 4

Devise a data structure that represents a multiset of n numbers under the following operations:

- Build: $\Theta(n)$
- Return median: $\Theta(1)$
- Delete median: $\Theta(\log(n))$

Question 4 – Solution

- We will maintain:
  - Max heap of the $\lceil n/2 \rceil$ smallest elements.
  - Min heap of the $\lfloor n/2 \rfloor$ largest elements.
- **Build:** find the median and partition in $\Theta(n)$, and then build a heap for each half separately. Time complexity: $\Theta(n)$.
- **Return median:** the median would be the root of the bigger heap (if n is even, both roots are medians). Time complexity: $\Theta(1)$.
- **Delete median:** if n is odd, delete the root of the first (bigger) heap. Otherwise, delete the root of the second heap (so it becomes smaller). Time complexity: $\Theta(\log(n))$